# Belema Lead of Engineering — Technical Assessment Answers

Oct 16, 2026 · @Oluwatosin

**Candidate:** Oluwatosin Akinyele · **Role:** Lead of Engineering, Belema

This submission answers all three questions. Each answer opens with its stated assumptions. The code for Question 2 is attached separately as `bank-search.zip`.

## Question 1 — End-to-end integration test module

**The brief.** Design an end-to-end integration test module that lives outside the Payment Gateway (PG). It must exercise the success and failure paths of three flows:

- **Flow A, card collection.** PG calls a card processor. A failed charge comes back as HTTP 200 with a failure code.
- **Flow B, collection by dynamic virtual account.** PG asks the Virtual Accounts service (VA) for an account; the customer pays in through the Switch; VA calls PG back.
- **Flow C, payout through the Switch.** PG debits the wallet, then calls the Switch, which calls the external bank. A failure (again HTTP 200) reverses the debit.

Deliverables are a whole-system block diagram, sequence diagrams for each flow, a design narrative and the assumed API contracts.

### Q1 assumptions

**Scope and environment**

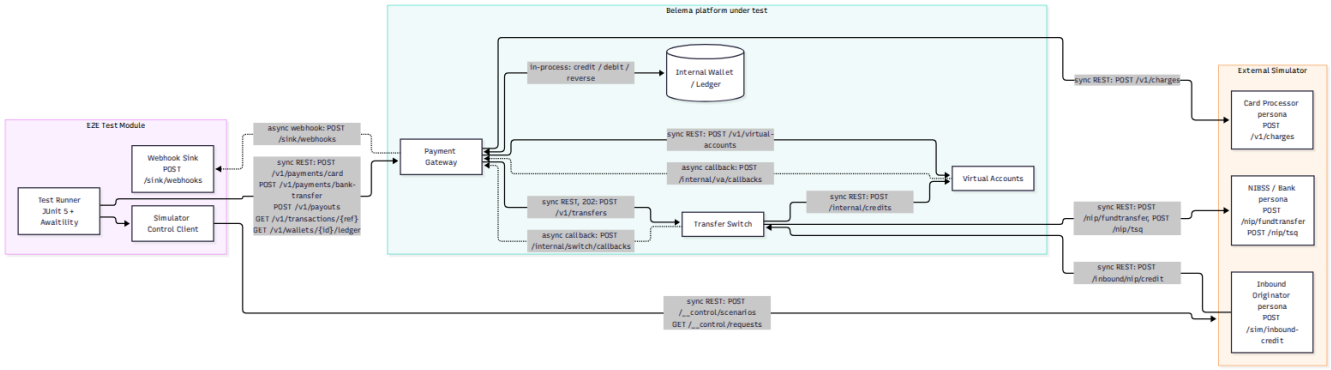
1. The test module is a separate repo and service, the **E2E Harness**, built with Java 21, JUnit 5, RestAssured, Awaitility, and Testcontainers or docker-compose.
2. PG, VA and the Switch run as real builds. All **external** parties (card processor, NIBSS or the destination bank, and the customer's bank paying in) are replaced by one programmable **Simulator**. Downstream URLs are set through configuration.
3. PG exposes a ledger read API that returns wallet entries filtered by reference.
4. PG notifies merchants by webhook. The harness registers itself as the test merchant's webhook URL.
5. Amounts are integers in kobo. Test merchants are set up with zero fees.
6. Mutating APIs accept an `Idempotency-Key`.

### API behaviour

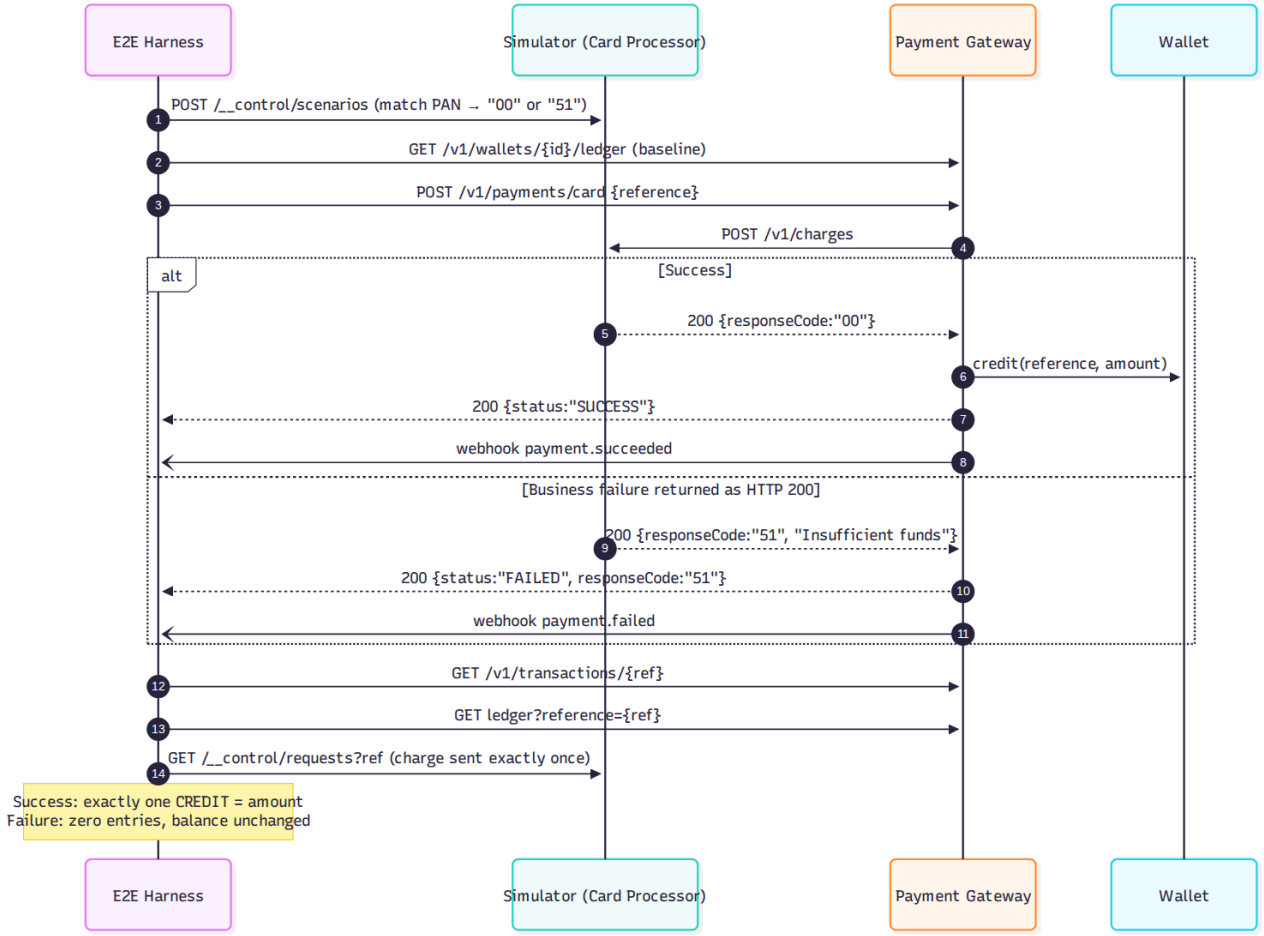
7. **Response codes.** Each integration has a documented, closed list of codes classed as SUCCESS, FAILED or PENDING, with `"00"` as success. An **unrecognised code or a malformed HTTP 200 body is treated as PENDING** and goes to review. It is never treated as success, so it causes no credit and no reversal.
8. **Transport failures are unknown outcomes, not failures.** A timeout, connection reset or 5xx response means we don't know what happened. In Flow C the payout stays PENDING, and the wallet is **not** reversed until the Switch resolves the status by status query (see Q3).
9. **Payout ledgering.** Flow C debits first, as the brief describes. A reversal is a separate ledger entry linked to the original debit through `reversalOf`, and it is idempotent on `transferRef`, so a debit is reversed at most once. A hold/reserve model would avoid the debit–reversal pair; the test design works with either.
10. **VA failure.** A Flow B failure callback means a credit arrived but was not acceptable: the amount didn't match (`VA02`) or the VA had expired (`VA03`). PG does not credit the wallet. Returning funds to the sender is handled outside PG and is out of scope.
11. **Callbacks.** They are HMAC-signed (`X-Signature`) and delivered at least once with retries, so they may be duplicated or out of order. Receivers de-duplicate on `eventId` or `sessionId`. Terminal states are final: a late SUCCESS arriving after FAILED is flagged for reconciliation, not applied.
12. **Switch interaction.** The Switch accepts `POST /v1/transfers` with **202** and reports the outcome to PG by **async callback**. It also exposes `GET /v1/transfers/{ref}`, which PG uses if the callback never arrives.
13. **Card flow.** The test environment has no 3DS or OTP step, and the processor returns a final result synchronously. Tests use the processor's test PANs only, so no real card data is involved.

## Whole-system block diagram

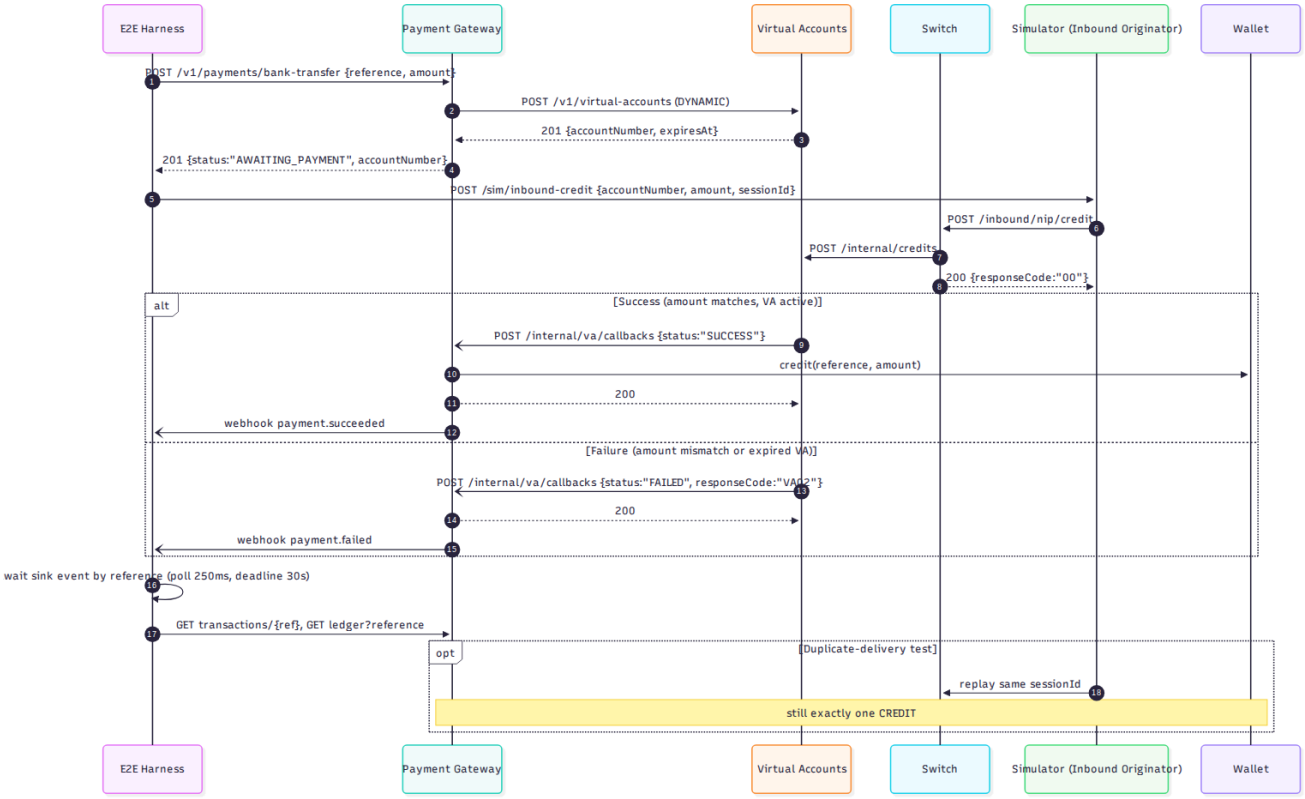
Solid arrows are synchronous REST calls; dotted arrows are asynchronous callbacks or webhooks.



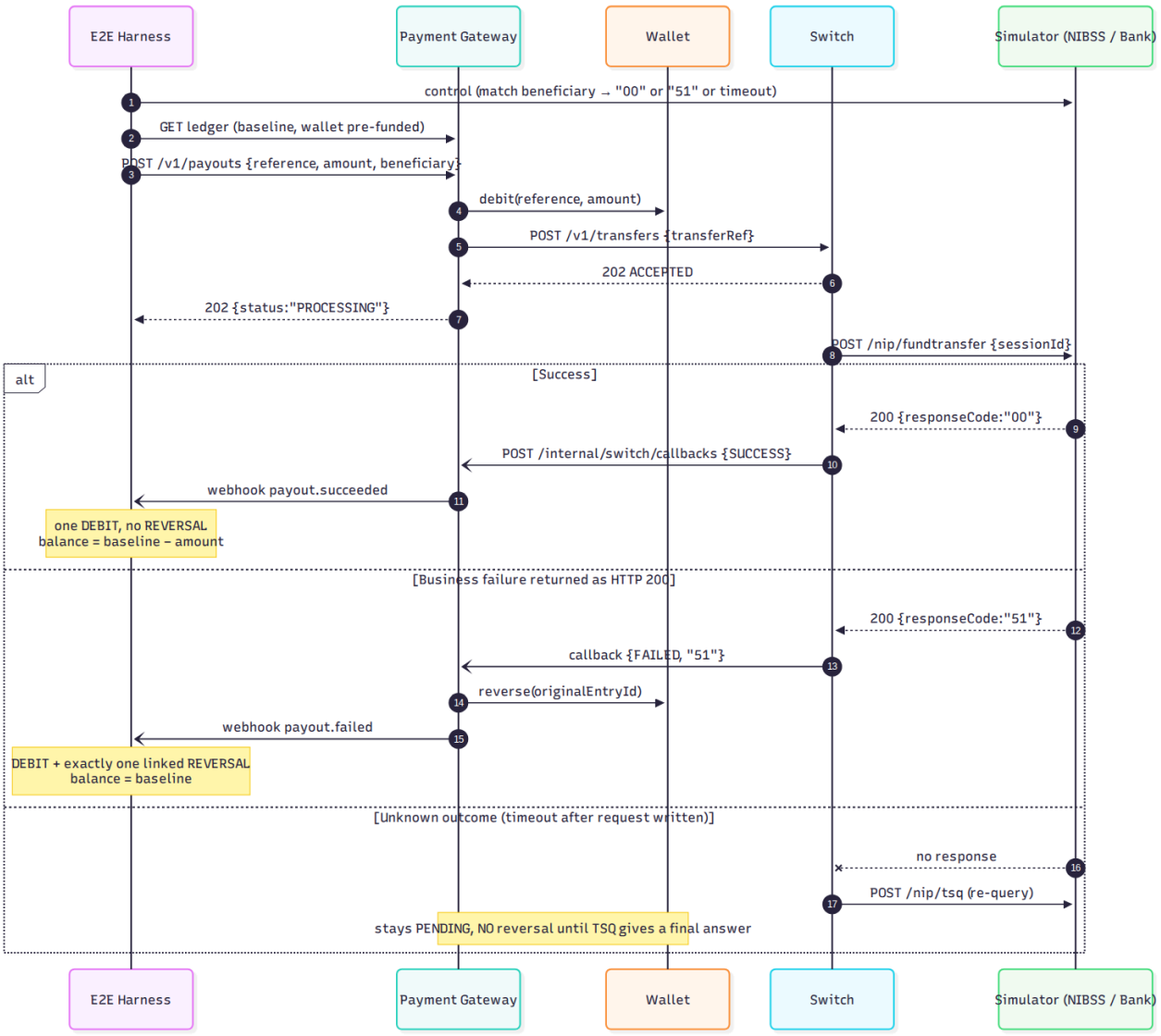

# Sequence diagram — Flow A, card collection



# Sequence diagram — Flow B, collection by dynamic VA



## Sequence diagram — Flow C, payout through the Switch




## Design narrative

**Invoking flows.** The harness acts as a merchant. It calls PG's public API with test keys and gives every test a unique `reference` (`e2e-{flow}-{uuid}`). That reference ties together the PG response, the webhook, the simulator's request log and the ledger entries. In Flow B the harness plays the paying customer: it tells the Inbound Originator persona to push a NIP credit into the Switch, so the real inbound path is exercised and nothing is shortcut into VA.

**HTTP 200 business failures.** The harness never asserts on the HTTP status code alone. Each test asserts three things together: the business `status`, the `responseCode`, and the wallet side effect. A table-driven test feeds every known processor and bank code

through the simulator and checks PG's mapping. Two cases in that table matter most:

- An **unrecognised code** must land as PENDING/REVIEW, never SUCCESS.
- A **200 response with a malformed or empty body** must not credit the wallet.

The most important assertion in the suite is that a 200 response carrying a failure code never moves money.

**Waiting for async callbacks without flakiness.** No test uses `sleep`. Each test awaits a condition with Awaitility, polling every 250ms against a hard 30s deadline. The condition is that a webhook for this reference has reached the sink **and** `GET /v1/transactions/{ref}` returns a terminal status. Requiring both channels to agree catches a webhook sent before the state was committed. Two choices keep this stable:

- Tests correlate by unique reference, so they can run in parallel without reading each other's events.
- The deadline comes from the service's SLA, not from tuning until the build goes green. On timeout, the harness dumps everything collected for that reference so the failure can be diagnosed at once.

**Controlling external dependencies.** The Simulator is configured per scenario, not through a global mode. A test registers a rule such as "match this PAN, beneficiary account or amount → respond with this result". Available results are:

- success, or a business failure with a given code
- a delay of N ms
- a timeout after the request is written
- a 5xx error
- a malformed body
- a duplicate or out-of-order callback

Rules are keyed on test data, so parallel tests can't interfere. The simulator also records every request it receives, which lets the harness check what our system *sent*: for example "charge attempted exactly once", or "TSQ issued after a timeout and the transfer was not re-sent".

**Proving wallet state.** The harness takes a ledger baseline before each test and checks the entries filtered by reference afterwards. A balance check alone isn't enough, because two wrong entries can cancel out and leave the balance looking right.

| Outcome | Expected ledger entries |
|---|---|
| Card or VA success | Exactly one CREDIT equal to the amount |
| Card or VA failure | No entries |
| Payout success | One DEBIT, no reversal |
| Payout failure | One DEBIT plus exactly one REVERSAL whose `reversalOf` is the DEBIT's id; net zero; balance equals baseline |
| Payout unknown outcome | One DEBIT, no REVERSAL while PENDING |

After every test the harness also checks that the sum of ledger entries equals the reported balance. Duplicate-callback tests confirm that the credit and the reversal each happen exactly once.

**Test data.** Each run provisions fresh merchants through an admin API, so every wallet starts empty and isolated. Payout tests fund the wallet through a real successful collection; a backdoor top-up would skip the flow under test. Test PANs, beneficiary accounts and bank codes come from a fixture table matching the simulator's rules. No data is shared between tests and nothing depends on test order.

**Environment isolation.** Every CI run gets its own ephemeral environment, either a docker-compose stack or a Kubernetes namespace. It holds PG, VA and the Switch built from the commit under test, their own databases, and the Simulator in place of all external parties. Nothing points at a real processor or at NIBSS; URLs come from configuration and stay inside the network. The VA expiry TTL is set short (about 60s) so expiry scenarios run without manipulating the clock.

**CI.** The pipeline runs: unit and consumer contract tests → build images → bring up the environment → health checks → E2E suite in parallel → collect artefacts → tear down. Artefacts are JUnit reports, simulator request logs, sink events and service logs. A smoke subset (one success and one failure per flow) gates every PR; the full matrix, including timeouts and duplicates, runs nightly and before release. Flaky tests are quarantined and fixed, never retried until they pass.

## Assumed API contracts

```
// PG — POST /v1/payments/card   (Idempotency-Key header)
{ "reference": "e2e-card-7f3a", "amount": 500000, "currency": "NGN",
  "card": { "pan": "5060990580000000001", "expiry": "12/28", "cvv": "123" } }
// 200
{ "reference": "e2e-card-7f3a", "status": "FAILED",
  "responseCode": "51", "responseMessage": "Insufficient funds" }

// Card processor (simulated) — POST /v1/charges → 200
{ "processorRef": "cp-99812", "responseCode": "51", "responseMessage":
"Insufficient funds" }

// PG — POST /v1/payments/bank-transfer → 201
{ "reference": "e2e-va-1c2d", "status": "AWAITING_PAYMENT",
  "virtualAccount": { "accountNumber": "9900012345", "bankName": "Belema
MFB",
                      "expiresAt": "2026-10-20T10:15:00Z" } }

// PG → VA — POST /v1/virtual-accounts
{ "type": "DYNAMIC", "merchantId": "m_e2e_01", "reference": "e2e-va-1c2d",
  "expectedAmount": 500000, "expiresAt": "2026-10-20T10:15:00Z",
  "callbackUrl": "https://pg.internal/internal/va/callbacks" }

// Simulator → Switch — POST /inbound/nip/credit
{ "sessionId": "000015261020101500123456789012", "destinationAccount":
"9900012345",
  "amount": 500000, "originatorName": "E2E Customer", "originatorBankCode":
"058" }

// VA → PG — POST /internal/va/callbacks   (X-Signature header)
{ "eventId": "evt_8812", "reference": "e2e-va-1c2d", "sessionId":
"0000152610...",
  "amount": 480000, "status": "FAILED", "responseCode": "VA02",
  "responseMessage": "Amount mismatch" }

// PG — POST /v1/payouts → 202
{ "reference": "e2e-po-5e6f", "amount": 200000,
  "beneficiary": { "accountNumber": "0123456789", "bankCode": "044", "name":
"Ada Obi" },
  "narration": "E2E payout" }
{ "reference": "e2e-po-5e6f", "status": "PROCESSING" }
```

```
// PG → Switch — POST /v1/transfers → 202
{ "transferRef": "e2e-po-5e6f", "amount": 200000, "destinationBankCode":
"044",
  "destinationAccount": "0123456789",
  "callbackUrl": "https://pg.internal/internal/switch/callbacks" }

// Switch → PG — POST /internal/switch/callbacks
{ "transferRef": "e2e-po-5e6f", "sessionId": "0000152610...", "status":
"FAILED",
  "responseCode": "51", "responseMessage": "Insufficient funds at
destination" }

// Switch — GET /v1/transfers/{transferRef}   (status query)
{ "transferRef": "e2e-po-5e6f", "status": "PENDING", "lastTsqAt": "2026-10-
20T10:02:30Z" }

// PG → merchant webhook (harness sink) — POST /sink/webhooks
{ "event": "payout.failed", "reference": "e2e-po-5e6f", "status": "FAILED",
  "responseCode": "51", "occurredAt": "2026-10-20T10:02:31Z" }

// PG — GET /v1/wallets/m_e2e_01/ledger?reference=e2e-po-5e6f
{ "balance": 1000000, "entries": [
  { "entryId": "le_1", "type": "DEBIT", "amount": 200000, "reference": "e2e-
po-5e6f" },
  { "entryId": "le_2", "type": "REVERSAL", "amount": 200000, "reference":
"e2e-po-5e6f",
    "reversalOf": "le_1" } ] }

// Simulator control — POST /__control/scenarios
{ "persona": "nibss", "match": { "destinationAccount": "0123456789" },
  "behaviour": { "fault": "TIMEOUT_AFTER_WRITE" }, "times": 1 }
```

## Question 2 — Lookahead search for bank names

**The brief.** Customers pick a destination bank from about 900 banks while adding a beneficiary. Build an in-house type-ahead (no Elasticsearch): choose and justify a data structure, implement insert and prefix suggestions, and state the complexity. Address case and punctuation, matching words other than the first, limiting and ordering results, and where the structure lives and how it is refreshed. Deliverables are source code with unit tests and a note under one page.

**Submitted code:** `bank-search.zip`, a Maven project in Java 17 with JUnit 5. Run it with `mvn test` (22 tests).

## Q2 assumptions

- The list has about 900 banks, each keyed by a unique NIP institution code. It changes rarely (weeks, not minutes), and the server owns it.
- Popularity means 30-day transfer counts across all customers. Each customer's own recent banks are a UI concern and out of scope here.
- Matching is word-prefix only. Typo tolerance is not required, and could be added later with edit-distance search over the same trie.
- Ops curates the aliases (GTB, FCMB, UBA).
- Names are in Latin script. The client is a mobile app, so the Java code ports directly to Kotlin.

## Choice of data structure

I used a **word-level prefix trie**. Every word of a bank's normalised name is a key, as is the whole name with spaces removed ("firstbankofnigeria") and each alias ("GTB", "FCMB"). Each trie node stores the set of bank codes that have a key starting with that node's prefix. A lookup walks the query's characters and reads the result straight off the node, with no subtree traversal.

### Alternatives considered

- *Linear scan using `contains()`.* Fast enough for 900 banks, but it does substring matching, so "ank" matches every "Bank", and the work per keystroke grows with the list.
- *Sorted array of (word, bankCode) pairs with binary search.* Equally valid and uses less memory. Lookup is O(log T + m), but it needs a range scan and de-duplication. The trie keeps lookup at O(L) and makes upsert and removal simple, and its extra memory (a few tens of thousands of references) is trivial.
- *Elasticsearch.* Ruled out by the brief, and not justified at this size.

## Complexity

N is the number of banks and T the total characters across all keys (about 60 per bank). L is the query length, q the number of query words, m the number of matches ($m \leqslant N$) and k the result limit.

| Operation | Cost |
|---|---|
| Insert one bank | O(t), where t is that bank's key characters |
| Build the index | O(T) |
| Remove or replace a bank | O(t) |
| Lookup | O(L + q·m + m log m) to walk, intersect and rank |
| Space | O(T) nodes plus O(T) code references, well under 1 MB |

Caching the top k at each node would give O(L + k) lookups. That isn't worth the rebuild it forces whenever popularity changes, given this data size.

## Points addressed

**Case, spaces and punctuation.** Names and queries go through the same normaliser. It strips accents ("Société" → "societe"), lower-cases with the root locale, turns "&" into "and", and drops apostrophes and full stops ("Joseph's" → "josephs", "U.B.A" → "uba"). Every other non-alphanumeric run, such as a space, hyphen or slash, separates words.

**Words other than the first.** Every word of the name is indexed, so "micro" finds every microfinance bank and "bank" finds most of the list. In a multi-word query, each word must prefix-match some word of the bank's name ("kuda micro"). Names typed without spaces ("firstbank") match through the space-free key.

**Limiting and ordering.** Results are ranked in this order:

1. Banks whose full name starts with the query, so "first ba" puts First Bank ahead of FCMB.
2. Popularity, highest first.
3. Alphabetical name, then code, so the order is deterministic.

The default limit is 10. In the app, the customer's own recent beneficiaries' banks appear above these suggestions.

**Where it lives and how it is refreshed.** It runs in both places, with the server as the authority.

- *Server.* Holds the canonical list (NIP institution list plus admin edits, with an active flag, aliases and popularity) and serves `GET /v1/banks` with a version and an ETag. The same library backs web and USSD search. When the list changes, the server builds a new index and swaps it in through an `AtomicReference`, so reads never lock.
- *Client.* The app ships with a bundled snapshot so the first launch works offline. On launch or foreground it makes a conditional GET with `If-None-Match`; normally that returns 304, and on 200 the app rebuilds its local index in milliseconds. Every keystroke is served locally with no network round trip.
- *Validation.* A stale client list can never send money to a deactivated bank, because the transfer API validates the bank code on the server.

### Code at a glance

| File | Role |
|---|---|
| `Bank.java` | Record: code, name, aliases, popularity |
| `BankNameNormalizer.java` | Case-, accent- and punctuation-insensitive normalisation and tokenising |
| `BankSearchIndex.java` | The trie: `build`, `insert` (upsert), `remove`, `suggest(query, limit)` |
| `BankNameNormalizerTest.java` | 5 tests on normalisation |
| `BankSearchIndexTest.java` | 17 tests: case, punctuation, accents, mid-word matches, aliases, ranking, ties, limits, upsert, removal, 900-bank build |

## Question 3 — Transaction lifecycle under uncertain outcomes

**The brief.** NIP transfers go to 22 destination banks. Bank X, the third-largest by volume, starts timing out on about 30% of requests *after* the request has been written, once the customer's wallet has been debited (or, for cards, an authorisation obtained). Nobody knows whether those beneficiaries were credited. Support holds 4,000 tickets raised in six hours, some customers claim non-receipt for transfers that did complete, and yesterday's settlement with Bank X is off by ₦180m in an unknown direction. Design the transaction lifecycle and failure handling so this is boring rather than catastrophic: states, timers, who makes the final call, reversal rules, idempotency, platform protection,

reconciliation, and one consistent status for support, customers and finance. Deliverables are a state diagram, a written design with timer values and reasoning, and the metrics and alerts that would have caught this early.

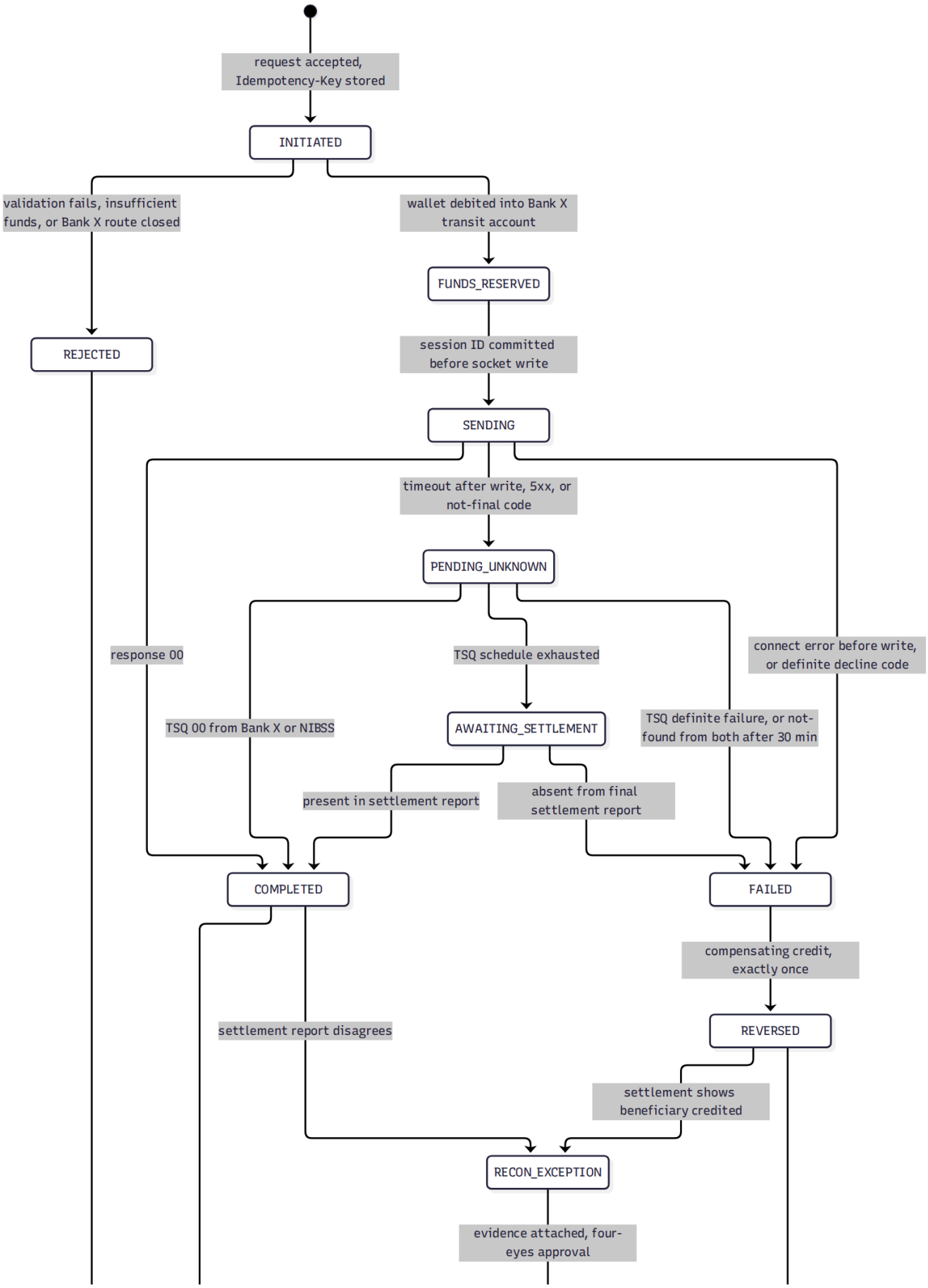
**The principle behind every rule below: a timeout is not a failure.** Money is reversed only on evidence that the credit did not happen, and one component alone decides a transfer's final status.

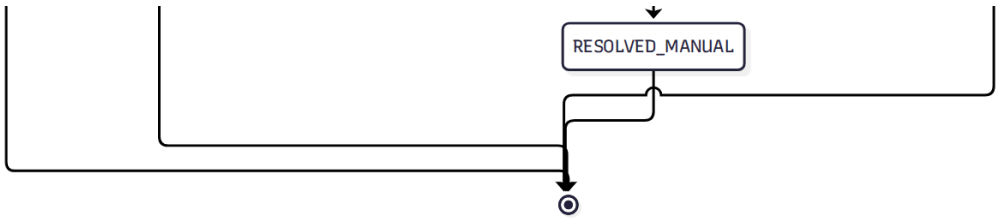
## Q3 assumptions

1. Our Switch reaches Bank X over a direct link and can also reach it through NIBSS NIP. Both Bank X and NIBSS support a transaction status query (TSQ) by session ID.
2. Every transfer gets a unique 30-character NIP session ID, generated and stored **before** anything is sent.
3. The NIBSS settlement report for each settlement session is the record of money that actually moved between banks.
4. Response codes follow the NIBSS NIP code list, kept as a classification table we maintain. Examples: `00` success; `03`, `05`, `07`, `51`, `57` definite failure; `01`, `09`, `25`, `96`, `97` not yet final.
5. The wallet uses double-entry ledgering, with a **transit account per destination bank** for money that has left the customer but is not yet confirmed at the beneficiary.
6. For card-funded transfers, the authorisation is held and captured only on success. The hold's validity (usually several days) is longer than our worst-case resolution time.
7. Baseline figures used to set thresholds are assumptions to confirm against real data: Bank X's normal p99 latency is about 3s and its normal timeout rate is below 0.5%.

## State machine

Transitions not drawn are forbidden. The arrows that matter most are the missing ones: there is **no arrow from PENDING_UNKNOWN to REVERSED**, and **no arrow from COMPLETED to FAILED**.


request accepted, Idempotency-Key stored
INITIATED
validation fails, insufficient funds, or Bank X route closed
wallet debited into Bank X transit account
REJECTED
FUNDS_RESERVED
session ID committed before socket write
SENDING
timeout after write, 5xx, or not-final code
PENDING_UNKNOWN
response 00
TSQ schedule exhausted
connect error before write, or definite decline code
TSQ 00 from Bank X or NIBSS
AWAITING_SETTLEMENT
TSQ definite failure, or not-found from both after 30 min
present in settlement report
absent from final settlement report
COMPLETED
FAILED
compensating credit, exactly once
settlement report disagrees
REVERSED
settlement shows beneficiary credited
RECON_EXCEPTION
evidence attached, four-eyes approval




| State | Meaning | Customer sees |
|---|---|---|
| INITIATED | Accepted and validated; no money moved | Processing |
| REJECTED | Refused before any debit, including when Bank X's route is closed | Not sent, money not taken |
| FUNDS_RESERVED | Customer debited; funds sit in Bank X's transit account | Processing |
| SENDING | Session ID committed; request being written | Processing |
| PENDING_UNKNOWN | Request reached Bank X, outcome unknown | Confirming with Bank X, money is safe |
| AWAITING_SETTLEMENT | TSQs exhausted; settlement report will decide | Confirming with Bank X, update by a stated time |
| COMPLETED | Beneficiary credited | Successful, with session ID as proof |
| FAILED | Proven not credited; reversal due | Failed, refund on the way |
| REVERSED | Refund posted | Failed, refunded |
| RECON_EXCEPTION | Settlement contradicts our final status | Under review (support contacts customer) |
| RESOLVED_MANUAL | Exception closed by finance with evidence | Final status as decided |

## Timers and thresholds

| Timer | Value | Reasoning |
|---|---|---|
| Connect timeout | 3s | A failure here means nothing was written, so the transfer is a definite non-delivery and safe to fail or reroute. |
| Read timeout | 30s | About 10× Bank X's normal p99. Long enough that slow but successful replies don't become unknowns; short enough to free connections. |
| First TSQ | 60s after send | Querying earlier mostly returns "record not found" while the bank is still processing, which looks like a failure but isn't. |
| Fast TSQ phase | +2, +3, +5, +10 min | Most unknowns resolve within minutes once the bank's back end catches up. |
| Slow TSQ phase | Every 15 min until 2h | Covers a bank that is degraded for a morning without flooding it. |
| Long TSQ phase | Hourly until 24h or the next settlement report | After this, AWAITING_SETTLEMENT and the report decides. |
| TSQ request timeout | 10s | A TSQ that times out is not an answer; it never moves the state. |
| "Not found" (`25`) rule | Final only if Bank X **and** NIBSS both return it, and the transfer is at least 30 min old | One source alone, or an early answer, is too often just lag. |
| Jitter | ±20% on every TSQ | Stops thousands of unknowns from querying in lockstep. |
| Manual queue | 4 business hours after the settlement report | For the rare transfer the report still doesn't settle. |

The 24-hour TSQ window sits inside the CBN timeline for reversing failed transfers, which sets the outer limit.

## Who makes the final call

One component, the **Transfer Resolver**, is the only writer of a transfer's status. Bank responses, TSQ results, settlement reports and ops decisions are all inputs to it, and it applies a fixed order of precedence.

| Rank | Source | What it can decide |
|---|---|---|
| 1 | NIBSS settlement report | Final. It records money that actually moved. Overrides everything below. |
| 2 | NIBSS TSQ | The central switch's record. Preferred over Bank X while Bank X is degraded. |
| 3 | Bank X TSQ | Authoritative when Bank X is healthy. |
| 4 | Original synchronous response | `00` or a definite decline is accepted; anything else leads to PENDING_UNKNOWN. |
| 5 | Ops decision | Only for RECON_EXCEPTION or an unresolved AWAITING_SETTLEMENT, with evidence and four-eyes approval. Never against rank 1. |

Two rules sit on top:

- **Success is sticky.** Once any credible source says `00`, a later failure from a lower-ranked source cannot undo it.
- **Conflicts never resolve silently.** A higher-ranked source contradicting a final state moves the transfer to RECON_EXCEPTION and raises an alert.

## Reversal rules

A reversal is allowed only when one of these holds:

- a definite decline code in the original response or a TSQ
- a connect failure, meaning the request was never written
- "not found" from both Bank X and NIBSS, at least 30 minutes after sending
- absence from the final settlement report covering the transfer

A reversal is **never** triggered by a read timeout, a 5xx, a TSQ timeout, an unclassified code, or a single early "not found".

Mechanically, a reversal is a compensating ledger entry from the Bank X transit account back to the customer, linked to the original debit. It carries the key `reversal:{transferId}` under a unique constraint, so it can be posted at most once whatever retries or duplicates arrive. For card-funded transfers the equivalent is voiding the authorisation; the hold stays in place while the outcome is unknown.

If a later settlement report shows a reversed transfer was in fact credited, it becomes a RECON_EXCEPTION. Recovery from the customer then follows the account terms, with notice. Each such case is an alert, and the target count is zero.

## Idempotency and duplicate prevention

- **Client requests.** Every transfer request carries an `Idempotency-Key`, stored with a hash of the request. A replay returns the original transfer instead of creating a new one.
- **Session ID before send.** The state SENDING and the session ID are committed in the same database transaction before the socket is written (outbox pattern). A crash mid-send therefore leaves a known session ID to query, never an orphan.
- **Unknown means query, never resend.** A PENDING_UNKNOWN transfer is resolved only by TSQ. Sending it again, through any route, risks paying the beneficiary twice.
- **Same session ID on any legitimate retry.** If a retry is ever allowed (for example after a connect failure), it reuses the session ID so the bank can de-duplicate. A `26` or `94` duplicate response means the original exists, and it triggers a TSQ rather than a failure.
- **Single writer.** State changes use compare-and-set on a version column, and only the Resolver performs them. Responses, TSQ results and report lines are processed idempotently, keyed by session ID.
- **Ledger.** Unique constraints on (transfer, entry type) make the debit, the transit release and the reversal each happen once.

## Protecting the platform while Bank X is degraded

- **Bulkheads.** Each destination bank has its own connection pool and in-flight limit (50 for Bank X). A slow Bank X cannot use up threads or connections needed for the other 21 banks.
- **Circuit breaker per bank.** It uses a 60-second rolling window with a minimum of 30 calls, and opens when timeouts plus 5xx reach 10%. At 30% timeouts it opens within about a minute.

- **Probing without moving money.** After 30 seconds open, the breaker goes half-open and probes Bank X with **name enquiries**, which carry no funds. It closes after 5 consecutive probes succeed under 2× normal p99 latency. A failed probe re-opens it with backoff (30s, 60s, 120s, capped at 5 minutes).
- **Routing new transfers while open.** Transfers not yet sent go to Bank X through NIBSS NIP instead of the direct link, if that path is healthy. If no healthy path exists, the transfer is **rejected before the debit** with a clear message, so the customer keeps their money and no new unknowns are created.
- **Never reroute an unknown.** Transfers already in PENDING_UNKNOWN stay on the TSQ schedule; they are never resent by another route.
- **TSQ load control.** While Bank X is degraded, TSQs go to NIBSS first. Queries to Bank X are capped at 10 per second, oldest and highest-value first, so recovery traffic doesn't prolong the outage.
- **Customers see it coming.** When the breaker opens, the app shows a Bank X delay notice on the transfer screen before the customer confirms.

## Reconciling the ₦180m break

The ledger design makes the direction visible. At send time the customer's money moves into Bank X's transit account. It moves to Bank X's settlement account on COMPLETED, or back to the customer on REVERSED. At any moment, the transit balance equals the value of Bank X transfers whose outcome is still unknown.

Reconciliation runs on every NIBSS settlement report, not once a day. It matches our ledger against the report **by session ID and amount** (never by amount alone) and puts every difference into one of these categories:

| Category | In settlement report | Our status | Direction | Action |
|---|---|---|---|---|
| A | Yes | PENDING_UNKNOWN | Timing only: customer debited, beneficiary credited | Resolve to COMPLETED |

| Category | In settlement report | Our status | Direction | Action |
|---|---|---|---|---|
| B | Yes | REVERSED | **We are short**: beneficiary credited and customer refunded | RECON_EXCEPTION; recover from customer |
| C | Yes | Not in our ledger | **We are short**: unexplained outflow | Critical investigation |
| D | No | COMPLETED | **We hold excess**: customer told success but money never moved | RECON_EXCEPTION; credit or refund |
| E | No | PENDING_UNKNOWN | We hold the customer's money in transit | Resolve to FAILED, then reverse |
| F | Yes, different amount | Any | Depends on sign | Investigate |

The break with Bank X is **settled value not recognised as COMPLETED by us, minus COMPLETED value not in the settlement report**. A positive result means NIBSS debited us more than we recognise, which is a potential loss; a negative result means we are holding money that belongs to customers. Finance receives the ₦180m as a signed total that splits into these categories, with a count, a value and a session-ID list for each, so the direction is known and every line has an owner.

Yesterday's break most likely sits in categories A and E: transfers in flight at the settlement cut-off whose status we hadn't resolved. In this design they are visible in the transit account before the report arrives, so the report confirms an expected number rather than revealing a surprise.

## One status for support, customers and finance

- **One source of truth.** The Resolver's status is served by one Status API that the app, the support console and the finance dashboard all read. Nobody checks Bank X separately and reaches a different answer.
- **Honest customer statuses.** Customers see the states in the table above. "Confirming with Bank X" says the money is safe and gives a time by which they will hear back.
- **Proactive notifications.** Every resolution sends a push and SMS straight away. Most tickets are customers asking a question we can answer before they ask.
- **Proof for non-receipt claims.** A COMPLETED transfer shows its session ID. The beneficiary quotes it to Bank X, which settles the claim without a ticket.
- **Support can trigger, not decide.** The console shows the full evidence trail (every response and TSQ, with source and time). Agents can request an immediate TSQ or escalate, but they cannot set a status.
- **Tickets linked to transfers.** Each ticket is tied to a transfer ID and closes itself with a templated reply when the transfer resolves. When the breaker opens, new Bank X tickets attach to one incident parent.
- **Finance sees the same thing.** Transit balance per bank, the pending list and the reconciliation categories all come from the same states.

The result is that the 4,000 tickets collapse to the RECON_EXCEPTION cases, which should be a handful.

## Metrics and alerts that would have caught this early

| Metric (per destination bank) | Alert | Why it matters |
|---|---|---|
| Timeout rate after write | Above 2% for 3 minutes (baseline below 0.5%) | Would have flagged Bank X within minutes, hours before the ticket spike. |
| p95 and p99 latency vs same hour last week | 3× baseline for 5 minutes | Latency usually rises before timeouts start. |
| Circuit breaker state change | Page on open | A bank-level incident has begun. |
| PENDING_UNKNOWN count, value and oldest age | Value above a set naira limit, or oldest above 30 minutes | Shows money at risk in real time. |

| Metric (per destination bank) | Alert | Why it matters |
|---|---|---|
| TSQ resolution rate within 10 minutes | Below 90% | TSQs aren't clearing the backlog, so settlement will have to. |
| Transit account balance vs pending total | Any mismatch | The ledger and the state machine disagree. |
| Success rate vs same hour last week | Drop of 5 points or more | Catches failures that don't show up as timeouts. |
| RECON_EXCEPTION count | Any new case | Each one is a possible loss or an owed refund. |
| Settlement break per session | Above a set naira limit | Finance learns within one session, not the next day. |
| Support tickets tagged by bank | Spike above baseline | A lagging signal, kept as a backstop. |